Machine Learning Techniques

Unit 3

Ankita Singh, Dept. of IT, AKGEC

Syllabus

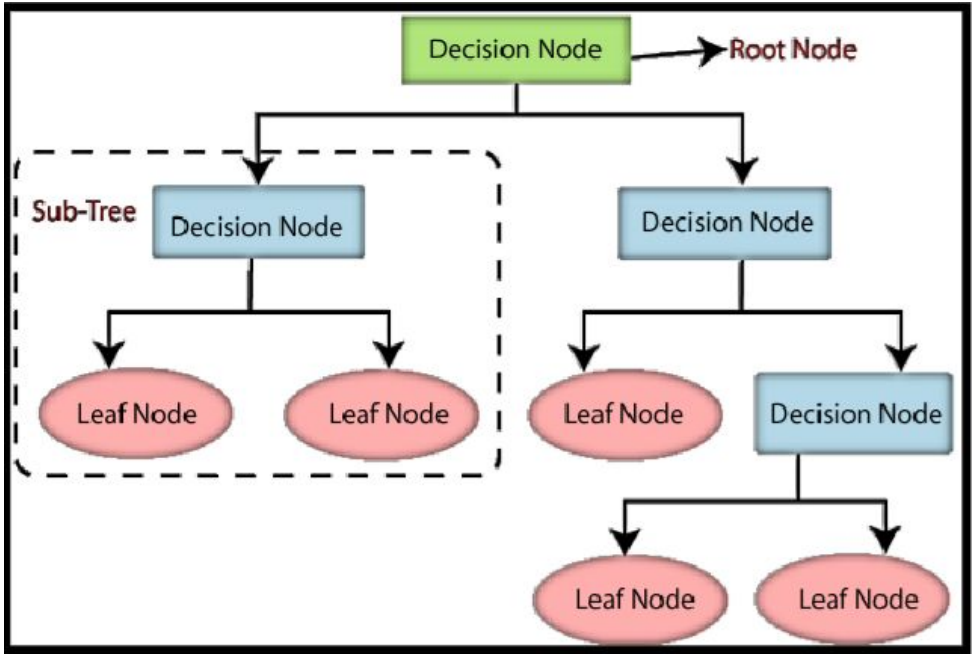
- DECISION TREE LEARNING - Decision tree learning algorithm, Inductive bias, Inductive inference with decision trees, Entropy and information theory, Information gain, ID-3 Algorithm, Issues in Decision tree learning.
- INSTANCE-BASED LEARNING – k-Nearest Neighbour Learning, Locally Weighted Regression, Radial basis function networks, Case-based learning.

Decision Tree

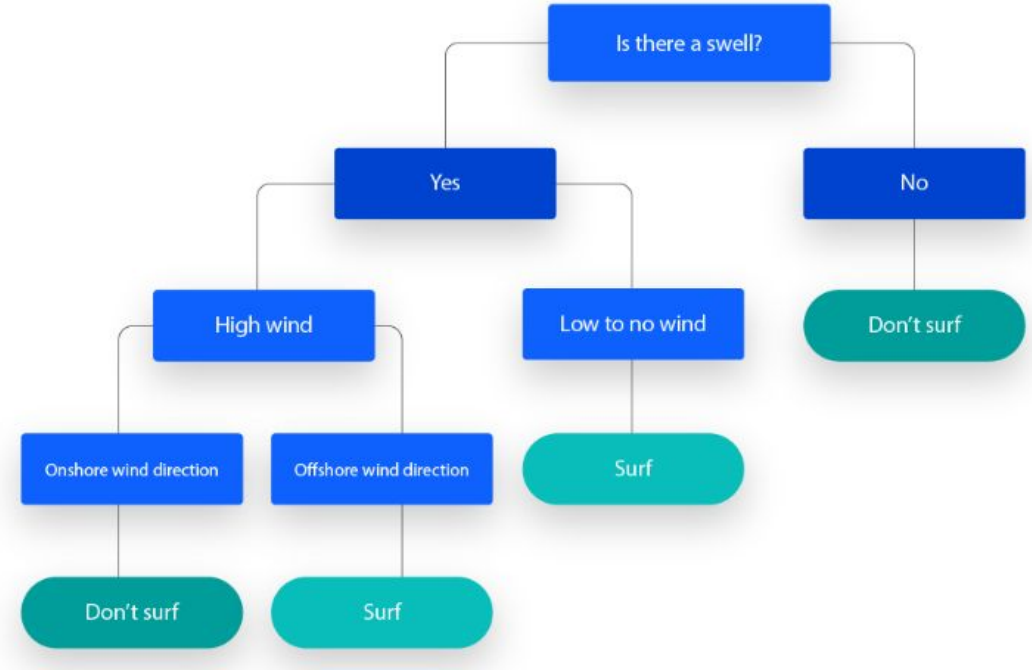
- Decision Tree is a Supervised learning technique that can be used for both classification and Regression problems, but mostly it is preferred for solving **Classification problems**.
- These models work by ***splitting data into subsets*** **based on** ***features*** this process is known as *decision making*. Each leaf node provides a prediction and the splits create a tree-like structure.
- It is a tree-structured classifier, where *internal nodes* represent the *features* of a dataset, *branches* represent the *decision rules* and each *leaf node* represents the *outcome*.
- Decision trees are popular because they are easy to interpret and visualize making it easier to understand the decision-making process.

- Decision Tree is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.
- It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.
- In a Decision tree, there are two nodes, which are the **Decision Node** and **Leaf Node**. Decision nodes are used to make any decision and have multiple branches,whereas Leaf nodes are the output of those decisions and do not contain any further branches.
- A decision tree can contain categorical data (YES/NO) as well as numeric data.

A decision tree simply asks a question, and based on the answer (Yes/No), it further split the tree into subtrees. Below diagram explains the general structure of a decision tree:



As an example, let's imagine that you were trying to assess whether or not you should go surf, you may use the following decision rules to make a choice:



Decision Tree Terminologies

• **Root Node**: Root node is from where the decision tree starts. It represents the entire dataset,which further gets divided into two or more homogeneous sets.

• **Leaf Node**: Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node.

• **Splitting:** Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.

• **Branch/Subtree**: A tree formed by splitting the tree.

• **Pruning**: Pruning is the process of removing the branches that split on features with low importance. It reduces complexity and prevent overfitting,

• **Parent/Child node**: The root node of the tree is called the parent node, and other nodes are called the child nodes.

How does the Decision Tree algorithm Work?

- Decision tree learning employs a divide and conquer strategy by conducting a greedy search to **identify the optimal split points** within a tree.

- This process of splitting is then repeated in a top-down, recursive manner until all, or the majority of records have been classified under specific class labels.

- Whether or not all data points are **classified as homogeneous set**s is largely dependent on the complexity of the decision tree.

- Smaller trees are more easily able to attain pure leaf nodes—i.e. data points in a single class.

- However, as a tree grows in size, it becomes increasingly difficult to maintain this purity, and it usually results in too little data falling within a given subtree. When this occurs, it is known as data fragmentation, and it can often lead to overfitting.

The complete process can be better understood using the below algorithm

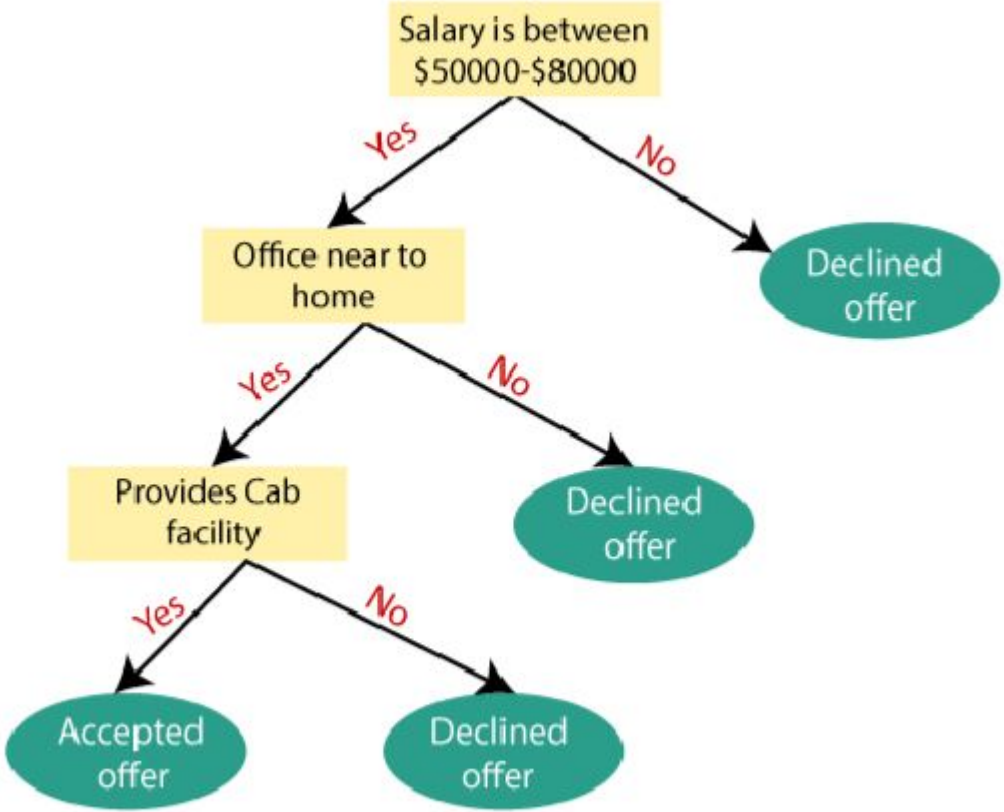
1. Find the best attribute in the dataset using Attribute Selection Measure(ASM) and make it as root node.
2. Break up the training set into subsets based on the branches of the root node.
3. Test the remaining attributes to see which ones fit best underneath the branches of the root node.
4. Continue this process for all other branches until:
    a. All the examples of a subset are of one type.
    b. There are no examples left.
    c. There are no more attributes left

Example

Example: Suppose there is a candidate who has a job offer and wants to decide whether he should accept the offer or Not.

So, to solve this problem, the decision tree starts with the root node (Salary attribute by ASM). The root node splits further into the next decision node (distance from the office) and one leaf node based on the corresponding labels. The next decision node further gets split into one decision node (Cab facility) and one leafnode. Finally, the decision node splits into two leaf nodes (Accepted offers and Declined offer). Consider the below diagram:

Salary_Range	Office_Near_Home	Cab_Facility	Offer_Status
In Range	Yes	Yes	Accepted offer
In Range	Yes	No	Declined offer
In Range	No	Yes	Declined offer
In Range	No	No	Declined offer
Out of Range	Yes	Yes	Declined offer
Out of Range	No	No	Declined offer


Salary is between $50000-$80000
Yes
No
Office near to home
Declined offer
Yes
No
Provides Cab facility
Declined offer
Yes
No
Accepted offer
Declined offer

Types of Decision Tree

- ID3: This algorithm measures how mixed up the data is at a node using something called entropy. It then chooses the feature that helps to clarify the data the most.
- C4.5 : This is an improved version of ID3 that can handle missing data and continuous attributes.
- CART: This algorithm uses a different measure called Gini impurity to decide how to split the data. It can be used for both classification (sorting data into categories) and regression (predicting continuous values) tasks.

Types of Decision Tree Algorithms

The different decision tree algorithms are listed below:

- ID3(Iterative Dichotomiser 3)
- C4.5
- CART(Classification and Regression Trees)
- CHAID (Chi-Square Automatic Interaction Detection)
- Random Forest
- XGBoost (Extreme Gradient Boosting)

Attribute Selection Measures

While implementing a Decision tree, the main issue arises that how to select the best attribute for the root node and for sub-nodes. So, to solve such problems there is a technique which is called as Attribute selection measure or ASM. By this measurement, we can easily select the best attribute for the nodes of the tree. There are two popular techniques for ASM, which are:

o Information Gain

o Gini Index

Entropy

Entropy is a concept that stems from information theory, which measures the impurity of the sample values. It is defined with by the following formula, where:

$$Entropy(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)$$

Where:

- S = the dataset (or subset) of samples
- c = number of classes
- p_i = proportion of samples in S that belong to class i

Entropy values can fall between 0 and 1 for 2 classes.

If all samples in data set, S, belong to one class, then entropy will equal zero. If half of the samples are classified as one class and the other half are in another class, entropy will be at its highest at 1. In order to select the best feature to split on and find the optimal decision tree, the attribute with the smallest amount of entropy should be used.

Information Gain:

- *Information gain* represents the **difference in entropy** before and after a split on a given attribute.
- The attribute with the highest information gain will produce the best split as it's doing the best job at classifying the training data according to its target classification.
- According to the value of information gain, we split the node and build the decision tree.

Information gain is usually represented with the following formula,

$$\text{Information Gain}(S,a) = \text{Entropy}(S) - \sum_{v \in values(a)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

where

a represents a specific attribute or class label

Entropy(S) is the entropy of dataset, S

$|Sv|/|S|$ represents the proportion of the values in S_v to the number of values in dataset, *S.*

Example

Day	Outlook	Temp	Humidity	Wind	Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Weak	Yes

$$P(\text{Yes}) = \frac{4}{7}, \quad P(\text{No}) = \frac{3}{7}$$

$$Entropy(S) = -\sum p_i \log_2(p_i)$$

$$Entropy(S) = -\left(\tfrac{4}{7}\log_2\tfrac{4}{7} + \tfrac{3}{7}\log_2\tfrac{3}{7}\right)$$

$$Entropy(S) = -(-0.461 - 0.523) = 0.984$$

Humidity	Tennis
High	No
High	No
High	Yes
High	Yes
Normal	Yes
Normal	No
Normal	Yes

Humidity = High (4 instances)

Tennis = Yes → 2 (Days 3, 4)

Tennis = No → 2 (Days 1, 2)

$$P(Yes) = 2/4 = 0.5, \quad P(No) = 0.5$$

$$Entropy(High) = -[0.5\log_2 0.5 + 0.5\log_2 0.5] = 1$$

(b) Humidity = Normal (3 instances)

- Tennis = Yes → 2 (Days 5, 7)
- Tennis = No → 1 (Day 6)

$$P(Yes) = 2/3, \quad P(No) = 1/3$$

$$Entropy(Normal) = -\left(\tfrac{2}{3}\log_2\tfrac{2}{3} + \tfrac{1}{3}\log_2\tfrac{1}{3}\right)$$

$$= -(0.667 \times -0.585 + 0.333 \times -1.585)$$

$$= 0.389 + 0.528 = 0.918$$

Weighted entropy after split

$$Entropy_{after}(Humidity) = \frac{4}{7}(1) + \frac{3}{7}(0.918)$$

$$= 0.571 + 0.393 = 0.964$$

Information Gain

$$Gain(S, Humidity) = Entropy(S) - Entropy_{after}(Humidity)$$

$$= 0.984 - 0.964 = 0.020$$

Gini Index

- Gini index is a measure of impurity or purity used while creating a decision tree in the CART(Classification and Regression Tree) algorithm.
- An attribute with the low Gini index should be preferred as compared to the high Gini index.
- It only creates binary splits, and the CART algorithm uses the Gini index to create binary splits.
- Gini index can be calculated using the below formula:

$$\text{Gini Impurity} = 1 - \sum_i (p_i)^2$$

ID3 Algorithm

- The ID3 algorithm was developed by Ross Quinlan in 1986. It builds a decision treeby recursively partitioning the dataset into smaller subsets until all data points in each subset belong to the same class.
- It employs a top-down approach, selecting features to split the dataset based on information gain.
- ID3 primarily deals with categorical properties, making it suitable for problems with discrete input features.
- One of its strengths is its ability to generate interpretable decision trees.
- However, ID3 can be sensitive to noisy data and prone to overfitting.

How Does ID3 Work?

The ID3 algorithm builds a decision tree in a top-down manner:

- Start with the root node representing the entire dataset.
- At each node, select the attribute that provides the most information gain about the target variable.
- Information gain measures how much uncertainty (entropy) is reduced by splitting based on a particular attribute.
- Recursively create child nodes for each possible value of the selected attribute.
- Continue until all data points in a subset belong to the same class or a stopping criterion is met.

Our Problem :

Name	Gender	Car Ownership	Travel Cost	Income Level	Transportation
Abhi	Male	1	Standard	High	?
Pavi	Male	0	Cheap	Medium	?
Ammu	Female	1	Cheap	High	?

Training Set

Attributes				Classes
Gender	Car Ownership	Travel Cost	Income Level	Transportation
Male	0	Cheap	Low	Bus
Male	1	Cheap	Medium	Bus
Female	1	Cheap	Medium	Train
Female	0	Cheap	Low	Bus
Male	1	Cheap	Medium	Bus
Male	0	Standard	Medium	Train
Female	1	Standard	Medium	Train
Female	1	Expensive	High	Car
Male	2	Expensive	Medium	Car
Female	2	Expensive	High	Car

Calculate the entropy of the total dataset

- First compute the Entropy of given training set.

Probability
Bus : 4/10 = 0.4
Train: 3/10 = 0.3
Car:3/10 = 0.3

$$E(S) = -P(I)\log_2 P(I)$$
$$E(S) = -(0.4)\log_2(0.4) - (0.3)\log_2(0.3) - (0.3)\log_2(0.3) = 1.571$$

Attributes				Classes
Gender	Car Ownership	Travel Cost	Income Level	Transportation
Male	0	Cheap	Low	Bus
Male	1	Cheap	Medium	Bus
Female	1	Cheap	Medium	Train
Female	0	Cheap	Low	Bus
Male	1	Cheap	Medium	Bus
Male	0	Standard	Medium	Train
Female	1	Standard	Medium	Train
Female	1	Expensive	High	Car
Male	2	Expensive	Medium	Car
Female	2	Expensive	High	Car

Attributes	Classes
Gender	Transportation
Male	Bus
Male	Bus
Female	Train
Female	Bus
Male	Bus
Male	Train
Female	Train
Female	Car
Male	Car
Female	Car

Split the dataset on 'Gender' attribute

Attributes	Classes
Gender	Transportation
Male	Bus
Male	Bus
Male	Bus
Male	Train
Male	Car

Probability
Bus : 3/5 = 0.6
Train: 1/5 = 0.2
Car:1/5 = 0.2

$E(S_v) = -0.6 \log_2(0.6) - 0.2 \log_2 (0.2) - 0.2 \log_2 (0.2)$
$= 1.522$

Attributes	Classes
Gender	Transportation
Female	Train
Female	Bus
Female	Train
Female	Car
Female	Car

Probability
Bus : 1/5 = 0.2
Train: 2/5 = 0.4
Car:2/5 = 0.4

$E(S_v) = -0.2 \log_2 (0.2) - 0.4 \log_2 (0.4) - 0.4 \log_2 (0.4)$
$= 1.371$

$Gain(S,A) = E(S) - I(S,A)$
$I(S,A) = 1.522 * (5/10) + 1.371 * (5/10)$

$Gain(S,A) = 1.571 - 1.447$
$= 0.12$

Attributes				Classes
Gender	Car Ownership	Travel Cost	Income Level	Transportation
Male	0	Cheap	Low	Bus
Male	1	Cheap	Medium	Bus
Female	1	Cheap	Medium	Train
Female	0	Cheap	Low	Bus
Male	1	Cheap	Medium	Bus
Male	0	Standard	Medium	Train
Female	1	Standard	Medium	Train
Female	1	Expensive	High	Car
Male	2	Expensive	Medium	Car
Female	2	Expensive	High	Car

Attributes	Classes
Car Ownership	Transportation
0	Bus
1	Bus
1	Train
0	Bus
1	Bus
0	Train
1	Train
1	Car
2	Car
2	Car

Split the dataset on 'ownership' attribute

Attributes	Classes
Car Ownership	Transportation
0	Bus
0	Bus
0	Train

Attributes	Classes
Car Ownership	Transportation
1	Bus
1	Train
1	Bus
1	Train
1	Car

Attributes	Classes
Car Ownership	Transportation
2	Car
2	Car

$$Gain(S,A) = E(S) - I(S,A)$$
$$I(S,A) = 0.918 * (3/10) + 1.522 * (5/10) + 0 * (2/10)$$

$$Gain(S,A) = 1.571 - 1.0364 = 0.534$$

Probability
Bus : 2/3= 0.6
Train: 1/3 = 0.3
Car:0/3 = 0
Entropy = 0.918

Probability
Bus : 2/5 = 0.4
Train: 2/5 = 0.4
Car:1/5 = 0.2
Entropy = 1.522

Probability
Bus : 0/2 = 0
Train: 0/2 = 0
Car:2/2 = 1
Entropy = 0

- If we choose Travel Cost as splitting attribute,

-Entropy for Cheap = 0.722

Standard = 0

Expensive = 0

IG = 1.21

- If we choose Income Level as splitting attribute,

-Entropy for Low = 0

Medium = 1.459

High = 0

IG = 0.695

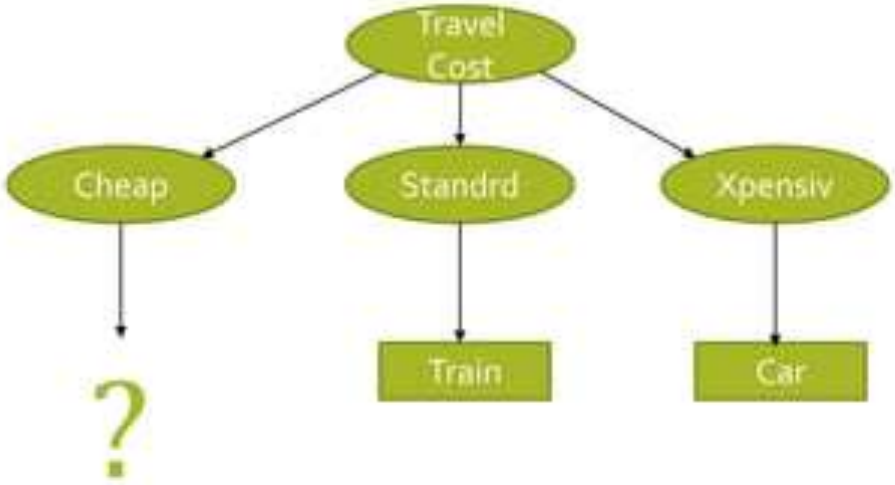
Attribute	Information Gain
Gender	0.125
Car Ownership	0.534
Travel Cost	1.21
Income Level	0.695

Attributes				Classes
Travel Cost	Gender	Car Ownership	Income Level	Transportation
Cheap	Male	0	Low	Bus
Cheap	Male	1	Medium	Bus
Cheap	Female	1	Medium	Train
Cheap	Female	0	Low	Bus
Cheap	Male	1	Medium	Bus

Attributes				Classes
Travel Cost	Gender	Car Ownership	Income Level	Transportation
Standard	Male	0	Medium	Train
Standard	Female	1	Medium	Train

Attributes				Classes
Travel Cost	Gender	Car Ownership	Income Level	Transportation
Expensive	Female	1	High	Car
Expensive	Male	2	Medium	Car
Expensive	Female	2	High	Car

Diagram : Decision Tree



Iteration on Subset of Training Set

Attributes				Classes
Travel Cost	**Gender**	**Car Ownership**	**Income Level**	**Transportation**
Cheap	Male	0	Low	Bus
Cheap	Male	1	Medium	Bus
Cheap	Female	1	Medium	Train
Cheap	Female	0	Low	Bus
Cheap	Male	1	Medium	Bus

Probability
Bus : 4/5 = 0.8
Train: 1/5 = 0.2
Car:0/5 = 0

$$E(S) = -P(I) \log_2 P(I)$$
$$E(S) = -(0.8) \log_2 (0.8) - (0.2) \log_2 (0.2) = 0.722$$

- If we choose Gender as splitting attribute,

-Entropy for Male = 0

Female = 1

IG = 0.322

- If we choose Car Ownership as splitting attribute,

-Entropy for 0 = 0

1 = 0.918

IG = 0.171

- If we choose Income Level as splitting attribute,

-Entropy for Low = 0

Medium = 0.918

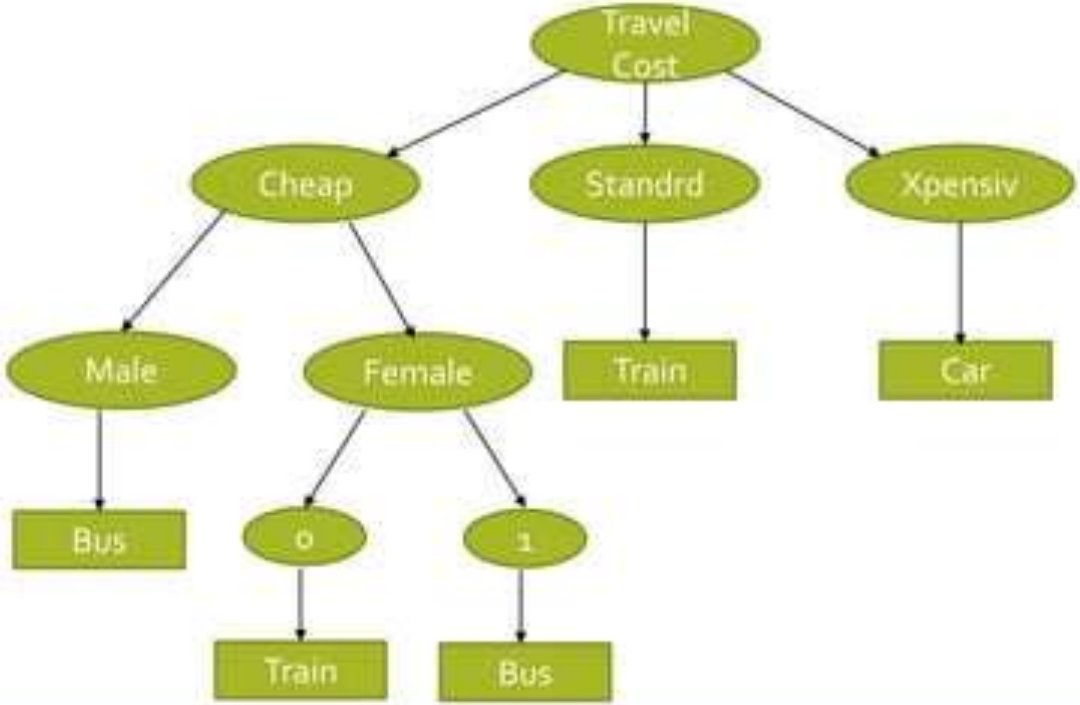
IG = 0.171

Attributes	Information Gain
Gender	0.322
Car Ownership	0.171
Income Level	0.171

Attributes			Classes
Gender	Car Ownership	Income Level	Transportation
Male	0	Low	Bus
Male	1	Medium	Bus
Male	1	Medium	Bus

Attributes			Classes
Gender	Car Ownership	Income Level	Transportation
Female	1	Medium	Train
Female	0	Low	Bus

Diagram : Decision Tree



Solution to Our Problem :

Name	Gender	Car Ownership	Travel Cost	Income Level	Transportation
Abhi	Male	1	Standard	High	Train
Pavi	Male	0	Cheap	Medium	Bus
Ammu	Female	1	Cheap	High	Bus

Drawbacks of ID3

- Data may be over-fitted or over-classified, if a small sample is tested.
- Only one attribute at a time is tested for making a decision.
- Classifying continuous data may be computationally expensive.
- Does not handle numeric attributes and missing values.

Advantages of Decision Tree:

1. **Interpretability**: Decision trees are easily interpretable and can be visualized, making them suitable for explaining the decision-making process to stakeholders and domain experts. The decision rules learned by decision trees are simple to understand and can be represented graphically.

2. **No Assumptions about Data Distribution**: Decision trees make no assumptions about the underlying distribution of the data, unlike parametric models such as linear regression. They can handle both numerical and categorical data without requiring data transformation.

3. **Implicit Feature Selection**: Decision trees automatically perform feature selection by selecting the most informative features at each split. Important features tend to appear closer to the root of the tree, making it easy to identify key predictors.

4. **Handling Non-Linear Relationships**: Decision trees can capture non-linear relationships between features and the target variable without the need for complex transformations or feature engineering. They can model complex decision boundaries with multiple splits.

Limitations of Decision Tree:

1. **Overfitting**: Decision trees are prone to overfitting, especially when the tree is deep or when the dataset is noisy or contains irrelevant features. Deep trees can capture noise in the training data, leading to poor generalization performance on unseen data.

2. **Instability**: Small changes in the training data can result in different tree structures,leading to high variance and instability. Decision trees are sensitive to the training data, and slight variations in the dataset can produce different trees.

3. **Bias Toward Dominant Classes**: Decision trees tend to favor splits that result in pure subsets, leading to a bias toward dominant classes in the dataset. Imbalanced Datasets may result in biased trees that perform poorly on minority classes

4. **Greedy Nature:** Decision trees use a greedy approach to construct the tree by selecting the best split at each node based on local information. This may not always lead to the globally optimal tree structure and may result in suboptimal solutions.

5. **Limited Expressiveness:** Decision trees may not be expressive enough to capture complex relationships in the data, especially when the decision boundaries are highly non-linear. Ensemble methods such as Random Forests or Gradient Boosting can be used to improve expressiveness.

Inductive Bias

In decision tree learning, the inductive bias refers to the assumptions and preferences encoded into the algorithm that guide the construction of the decision tree.

These biases influence the choice of attributes for splitting, the structure of the resulting tree, and the generalization capabilities of the model.

Here's how inductive bias manifests in decision tree learning:

1. Bias towards Simplicity:

Decision tree algorithms have a bias towards simpler explanations that can be easily understood and interpreted by humans. This bias leads to the preference for smaller, more interpretable trees over larger, more complex ones.

2. Attribute Selection Bias:

Decision tree algorithms use a greedy approach to select attributes for splitting at each node. This bias towards local optimization may lead to suboptimal trees, as the algorithm chooses the best attribute at each step without considering future splits.

3. Bias towards High Information Gain:

Decision tree algorithms, such as ID3 and C4.5, select the attribute that maximizes information gain or another splitting criterion (e.g., Gini impurity or gain ratio) at each node. This bias towards maximizing information gain helps to partition the data into more homogeneous subsets, leading to a better separation of classes.

4. Bias towards Small Trees:

Decision tree algorithms have an inductive bias towards smaller trees to avoid overfitting. They aim to find the simplest tree that adequately captures the underlying patterns in the data. This bias is often implemented through tree pruning techniques that remove unnecessary branches or nodes from the tree.

5. Bias towards Majority Class:

In the absence of a clear split based on the selected attribute, decision tree algorithms may bias towards the majority class. This bias ensures that the resulting tree makes a prediction based on the most prevalent class in the subset, reducing the risk of misclassification.

Importance of Inductive Bias in Decision Trees:

Interpretability: The bias towards simplicity and small trees makes decision trees highly interpretable, allowing users to understand the decision-making process and the factors influencing predictions.

Generalization: The bias towards high information gain and smaller trees helps decision trees generalize well to unseen data, leading to robust and accurate predictions.

Efficiency: The bias towards attribute selection and local optimization ensures that decision tree algorithms are computationally efficient, making them suitable for large datasets and real-time applications.

Instance Based Learning

- In contrast to learning methods, that construct a general, explicit description of the target function when training examples are provided, instance based learning constructs the target function only when a new instance must be classified.
- Each time a new query instance is encountered, its relationship to the previously stored examples is examined in order to assign a target function value for the new instance.
- It's a flexible approach that can handle evolving datasets and provides a different perspective from traditional model-based methods.

- Instance based methods are sometimes referred to as **lazy learning methods** because they delay processing until a new instance must be classified.
- A key advantage of this kind is delayed, or lazy, learning is that instead of estimating the target function once for entire instance space, these methods can estimate it locally and differently for each new instance to be classified.

Comparison between lazy learning (instance-based learning) and eager learning (model-based learning)

Feature	Lazy Learning	Eager Learning
Training Approach	Memorizes entire training data during training.	Creates a generalized representation during training.
Prediction Process	Searches for similar instances during prediction and applies their labels.	Directly applies the learned representation for prediction.
Adaptability to New Data	Quickly adapts to new data without retraining.	Less adaptable to new data; retraining might be necessary.
Prediction Speed	It can be slower, especially with large datasets.	Faster predictions due to pre-trained model.
Offline Predictions	Requires access to training data during prediction.	Can make predictions offline or without training data.
Handling Complex Relationships	Effective at handling complex and non-linear relationships.	Better suited for well-defined patterns and relationships.
Model Representation	No fixed model representation; relies on memorized data.	Requires a fixed model representation from training.

Principles of Instance-Based Learning:

1. **Data Memorization**: Instance-based learning algorithms memorize the entire training dataset, storing it for future reference during the prediction phase.
2. **Lazy Learning**: Instance-based learning is often referred to as lazy learning because it defers the processing of training data until a new instance needs to be classified or predicted.
3. **Similarity Measure:** Instance-based learning relies on a similarity measure (distance metric) to find the most similar instances in the training data to the new instance being classified.
4. **Local Decision Making:** Instead of building a global model, instance-based learning makes decisions based on the local neighborhood of similar instances in the feature space.

Advantages of Instance-Based Learning:

- **Flexibility:** Instance-based learning can handle *complex decision boundaries and non-linear relationships* between features, as it directly uses the training instances for prediction.
- **Adaptability**: Instance-based learning can *adapt quickly to changes* in the data distribution, as the model does not need to be retrained when new data becomes available.
- **Interpretability:** Instance-based learning provides transparency, as predictions are based on the actual training instances, making it easier to understand and interpret the model's behavior.

Limitations of Instance-Based Learning:

- **High Memory Requirements:** Storing the entire training dataset can be memory-intensive, especially for large datasets with many instances and features.
- **Computational Complexity:** Making predictions with instance-based learning can be computationally expensive, as it requires calculating distances between the new instance and all training instances.
- **Sensitivity to Noise:** Instance-based learning can be sensitive to noisy data, outliers, and irrelevant features, as it directly uses the training instances for prediction.

Applications of Instance-Based Learning:

1. **Classification:** k-Nearest Neighbors is commonly used for classification tasks, such as text categorization, image recognition, and medical diagnosis.
2. **Regression:** k-Nearest Neighbors can also be applied to regression tasks, such as predicting house prices or stock prices based on similar historical instances.
3. **Anomaly Detection:** Instance-based learning can be used for anomaly detection, where rare or unusual instances are identified based on their dissimilarity to normal instances.

Algorithms in Instance-Based Learning:

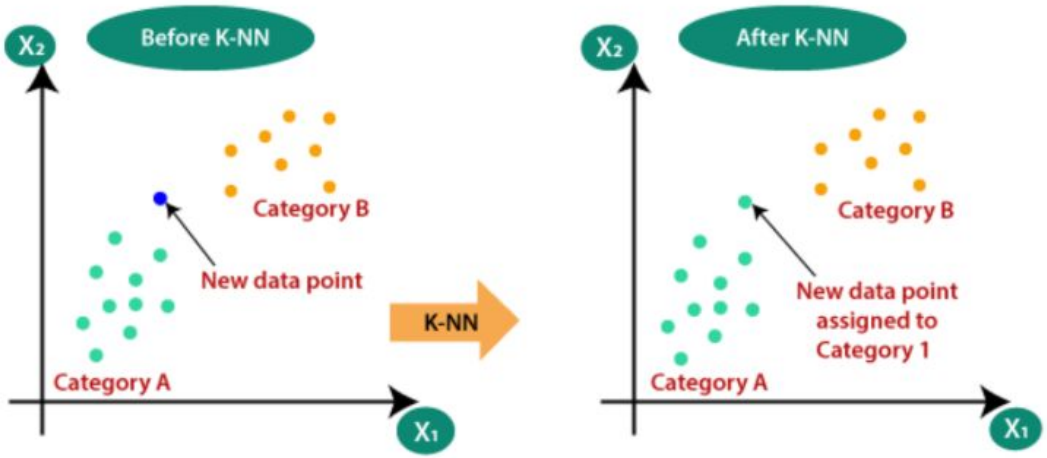
1. **k-Nearest Neighbors (k-NN):** One of the most popular instance-based learning algorithms. Given a new instance, k-NN finds the k nearest neighbors in the training data and assigns the majority class label (for classification) or averages the target values (for regression).
2. **Case-Based Reasoning (CBR):** CBR is a problem-solving paradigm that relies on past experiences (cases) to solve new problems. It stores a database of cases and retrieves the most similar cases to the current problem to make decisions.
3. **Locally Weighted Learning (LWL):** Assigns different weights to training instances based on their proximity to the query instance.
4. **Radial basis function networks**

K-Nearest Neighbour (KNN) Algorithm

- K-Nearest Neighbors (KNN) is a supervised machine learning algorithm generally used for classification but can also be used for regression tasks.
- This algorithm assumes all instances corresponds to points in n-dimensional space.
- It works by finding the "k" closest data points (neighbors) to a given input and makes a predictions based on the majority class (for classification) or the average value (for regression).
- The nearest neighbor of an instance are defined in terms of the standard euclidean distance.
- Since KNN makes no assumptions about the underlying data distribution it makes it a non-parametric and instance-based learning method.

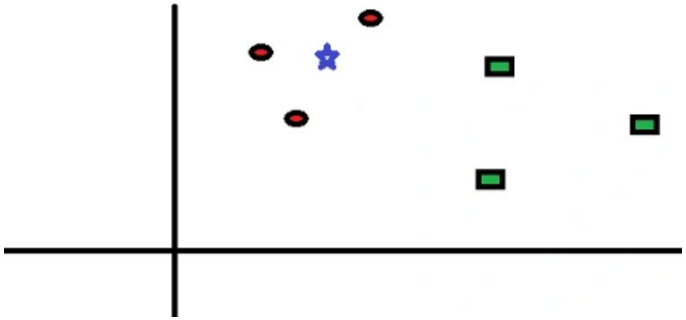
Why do we need a K-NN Algorithm?

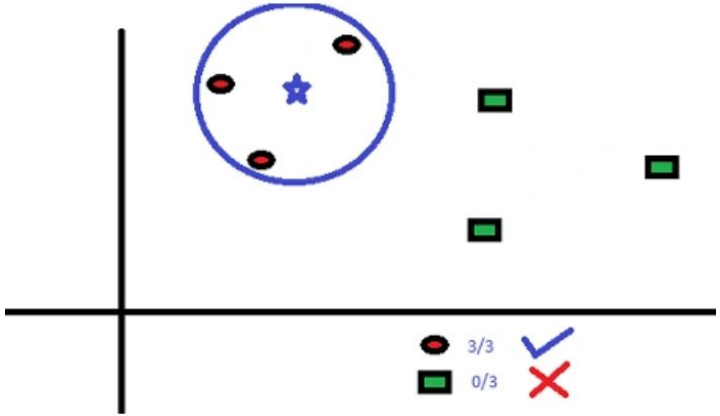
Suppose there are two categories, i.e., Category A and Category B, and we have a new data point x1, so this data point will lie in which of these categories. To solve this type of problem, we need a K-NN algorithm. With the help of K-NN, we can easily identify the category or class of a particular dataset. Consider the below diagram:



How does K-NN work?

- Let's take a simple example. Following is a spread of red circles (RC) and green squares (GS)
- We intend to find out the class of the blue star (BS). BS can either be RC or GS and nothing else. The "K" in KNN algorithm is the nearest neighbor we wish to take the vote from.
- Let's say K = 3. Hence, we will now make a circle with BS as the center just as big as to enclose only three data points on the plane. Refer to the following diagram for more details:
- The three closest points to BS are all RC. Hence, with a good confidence level, we can say that the BS should belong to the class RC.
- Here, the choice became obvious as all three votes from the closest neighbor went to RC.
- The choice of the parameter K is very crucial in this algorithm.





The KNN algorithm follows a straightforward, step-by-step approach:

Step 1: Determine the Number of Nearest Neighbors (k)

The first step is to select the number of neighbors (**k**) to consider. The value of k determines how many neighboring points will influence the classification or prediction of a new data point.

Step 2: Calculate the Distance Between the Query Point and Dataset Points

For each data point in the dataset, the algorithm calculates the distance between the query point (the new point to be classified or predicted) and every other point. Various distance metrics can be used, such as **Euclidean distance**, **Manhattan distance**, or **Minkowski distance**.

Step 3: Sort and Select the k-Nearest Neighbors

After calculating the distances, the algorithm sorts all data points in ascending order of distance. It then selects the **k-nearest neighbors**—the data points that are closest to the query point.

Step 4: Make a Prediction

- **For classification**: The algorithm assigns the query point to the class label that is most frequent among the k-nearest neighbors (majority voting).
- **For regression**: The algorithm predicts the value by averaging the values of the k-nearest neighbors.

How to Select the Value of K in the K-NN Algorithm?

Choosing the correct value of **k** is critical to the performance of the KNN algorithm. A small k value can make the model too sensitive to noise, resulting in **overfitting**, while a large k value can oversimplify the model, causing **underfitting**.

Methods to Select the Optimal k:

- **Cross-Validation**: A commonly used technique for choosing the value of k is cross-validation. By splitting the dataset into training and validation sets and evaluating model performance across different values of k, the optimal k value can be determined based on which k produces the lowest error rate.
- **Common Values of k**: In practice, values of k such as 3, 5, or 7 are typically chosen. Smaller values of k allow the model to capture local patterns, while larger k values generalize better across the dataset.

Impact on Model Performance:

- **Too small k (e.g., k = 1)**: The model is highly sensitive to individual data points, leading to overfitting, as the prediction is based on just one point.
- **Too large k**: When k is too large, the model can become too smooth, losing important patterns in the data, leading to underfitting.

In summary, selecting an appropriate k value ensures the balance between model complexity and predictive accuracy.

Distance Metrics Used in KNN Algorithm

Distance metrics are crucial for calculating the similarity between data points in KNN. Here are the commonly used metrics:

1. Euclidean Distance: Euclidean distance is the most common distance metric, calculated as the straight-line distance between two points in Euclidean space. For two points and , the Euclidean distance is calculated as:

$$d(P, Q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Euclidean distance is suitable for continuous variables and is easy to compute, making it a popular choice in KNN.

2. Manhattan Distance: Manhattan distance (or L1 distance) measures the distance between two points along the axes at right angles. For two points and , the Manhattan distance is calculated as:

$$d(P, Q) = |x_2 - x_1| + |y_2 - y_1|$$

It is useful for grid-like paths (e.g., city blocks) and is often employed when variables are more discrete.

Example

- Given the following training instances (see table), each having two attributes (x1 and x2). Compute the class label for test instance t1 = (3,7) using three-nearest neighbors (k=3).

Training Instance	x_1	x_2	Output
I_1	7	7	0
I_2	7	4	0
I_3	3	4	1
I_4	1	4	1

Training Instance	x_1	x_2	Output	Distance	Neighbor Rank
I_1	7	7	0	$\sqrt{(7-3)^2+(7-7)^2}=4$	3
I_2	7	4	0	$\sqrt{(7-3)^2+(4-7)^2}=5$	4
I_3	3	4	1	$\sqrt{(3-3)^2+(4-7)^2}=3$	1
I_4	1	4	1	$\sqrt{(1-3)^2+(4-7)^2}$ $=3.6$	2

k=3 so nearest neighbours are I_3, I_4, I_1

According to I_3 , I_4 class=1 and I_1 class=0

Answer is class =1

Assignment

Sepal Length	Sepal Width	Species
5.3	3.7	Setosa
5.1	3.8	Setosa
7.2	3.0	Virginica
5.4	3.4	Setosa
5.1	3.3	Setosa
5.4	3.9	Setosa
7.4	2.8	Virginica
6.1	2.8	Verscicolor
7.3	2.9	Virginica
6.0	2.7	Verscicolor
5.8	2.8	Virginica
6.3	2.3	Verscicolor
5.1	2.5	Verscicolor
6.3	2.5	Verscicolor
5.5	2.4	Verscicolor

Sepal Length	Sepal Width	Species
5.2	3.1	?

Advantages:

1. **Simplicity:** k-NN is easy to understand and implement, making it suitable for beginners.
2. **Non-parametric**: It makes no assumptions about the underlying data distribution, making it versatile and adaptable to various types of data.
3. **Flexibility:** k-NN can handle multi-class classification and regression tasks.
4. **Locally Adaptive**: It can capture complex decision boundaries and adapt to the local structure of the data.

Limitations:

1. **Computational Complexity**: As the size of the training dataset grows, the computational cost of finding the nearest neighbors increases.
2. **Memory Usage**: k-NN requires storing the entire training dataset in memory, which can be memory-intensive for large datasets.
3. **Sensitive to Noise**: It can be sensitive to noisy or irrelevant features, as it relies on the similarity between instances.
4. **Curse of Dimensionality**: Performance may degrade in high-dimensional spaces due to the increased sparsity of data points.

Applications:

1. Classification: k-NN is used in various domains such as image recognition, text categorization, and medical diagnosis.

2. Regression: It can be applied to regression tasks such as predicting house prices, stock prices, or weather forecasts.

3. Anomaly Detection: k-NN can be used for outlier detection or anomaly detection tasks.

Locally Weighted Regression

The phrase "**locally weighted regression**" is called

- **local** because the function is approximated based only on data near the query point,
- **weighted** because the contribution of each training example is weighted by its distance from the query point, and
- **regression** because this is the term used widely in the statistical learning community for the problem of approximating real-valued functions.

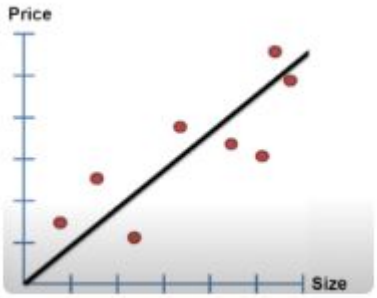
Locally Weighted Regression

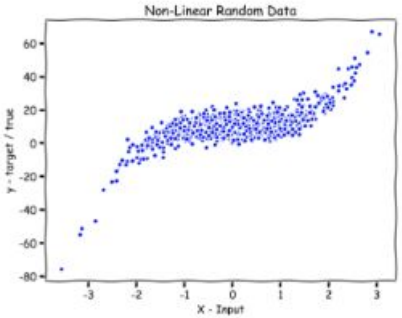
- Locally Weighted Linear Regression (LWLR) is a non-parametric regression technique that aims to fit a linear regression model to a dataset by giving more weight to nearby data points.
- Unlike traditional regression methods that fit a global model to the entire dataset, LWR fits a separate model to each data point, giving more weight to points that are closer to the point being predicted.
- For example, consider a dataset of temperature readings and corresponding energy consumption.
- LWLR can be used to predict the energy consumption for a given temperature reading by fitting a linear regression model to the training data, where the weight assigned to each training data point is inversely proportional to its distance from the query point.
- This means that training data points that are closer to the query point will have a higher weight and contribute more to the linear regression model.

- LWLR fits a separate linear regression model for each query point based on the weights assigned to the training data points.
- The weights assigned to each training data point are inversely proportional to their distance from the query point.
- Training data points that are closer to the query point will have a higher weight and contribute more to the linear regression model.
- LWLR is useful when a global linear model does not well-capture the relationship between the input and output variables. The goal is to capture local patterns in the data.

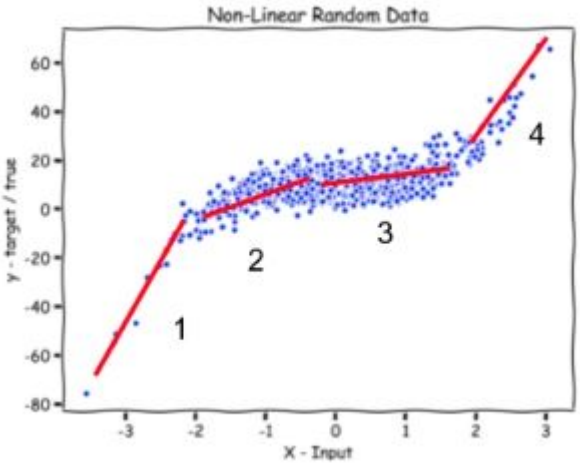
Why Locally Weighted Linear Regression Algorithm?

- In the traditional linear regression algorithm, we aim to fit a general line on the given training dataset and predict some parameters that can be generalized for any input to make correct predictions.
- Say we have a non-linear dataset, as shown in the image. A standard line wouldn't fit entirely into this type of dataset.
- In such a case, we have to introduce a locally weighted algorithm that can predict a very close value to the actual value of a given query point.
- We can break the given dataset into a smaller dataset and say we have multiple smaller lines that can fit the smaller dataset. Together, these multiple lines fit the entire dataset.





- Look at the 4 small lines that have been tried to fit smaller datasets and fit the entire dataset together.
- But now, the question arises of how to select the best line that can be chosen to predict the output for a given query point.
- Before we move on to find the best fit line, we must understand that we will always learn a different parameter for a particular query point.
- Hence locally weighted regression is a non-parametric algorithm.



Assigning weights to a line

We assume that there is a corresponding weight for that particular line given a query point. The one with the greatest weight will be the best fit line and will be used for predicting the output of a given query point.
We know that if a given query point is close to a point on a particular line, it should have a greater weight than a far from the query point. We define a weight function which is given as,

$$W^{(i)} = \exp\left(-\frac{(x^{(i)} - x)^2}{2\tau^2}\right)$$

$x^{(i)}$ = any general training point,
x = given query point and,
τ = is bandwidth parameter

The bandwidth parameter controls how quickly the weight should fall with the distance of the point with the training point. In simpler terms, it controls the width of how varied the data is.
You can find the resemblance of the above function with the gaussian function.
Now assuming we fix the bandwidth parameter, we see that if we are given a query point close to a general training point, the weight would be close to 1 as the difference is close to 0. Similarly, if it's too far from the training point, then the weight would be close to 0. Hence we choose this as our weight function.

Case-Based Reasoning

Instance-based methods such as k-NEAREST NEIGHBOR and locally weighted regression share three key properties.

First, they are lazy learning methods in that they defer the decision of how to generalize beyond the training data until a new query instance is observed.

Second, they classify new query instances by analyzing similar instances while ignoring instances that are very different from the query.

Third, they represent instances as real-valued points in an n-dimensional Euclidean space.

- In **CBR** the instances are not represented as real-valued points, but instead, they use a **rich symbolic representation** and the methods used to retrieve similar instances are correspondingly more elaborate.

- **CBR** has been applied to problems such as
  – conceptual design of mechanical devices based on a stored library of previous designs,
  – reasoning about new legal cases based on previous rulings.

What is Case-Based Reasoning?

- Case-Based Reasoning is a technique that **solve problems** based on **past experiences**.
- The technique is derived from human problem-solving approaches, where people often rely on their past experiences to make decisions in new situations.
- CBR is a type of machine learning that utilizes a database of previously solved problems or cases to solve new problems.
- CBR is based on the idea that similar problems can have similar solutions, and it uses this similarity to find solutions to new problems.

Key terms in CBR

- **Case**: A case is a problem that has been previously solved and stored in the database.
- **Similarity:** The similarity measure is used to determine the degree of resemblance between past cases and the current situation.
- **Adaptation**: Adaptation is the process of modifying a retrieved past case to fit the current situation.

Process in Case-Based Reasoning

The CBR process typically involves four main steps: retrieve, reuse, revise, and retain.

- **Retrieve- Given a new case, retrieve similar cases from case base**

The first step in the CBR process is to retrieve relevant cases from a case library. This involves searching through the library to find cases that are similar to the current problem. The goal is to identify cases that are as close to the current problem as possible, as these are the most likely to provide useful information.

- **Reuse- Adapt the retrieved cases to fit the new case**

Once relevant cases have been retrieved, the next step is to reuse them to solve the current problem. This involves **adapting the solutions** used in past cases to fit the current problem. The goal is to find a solution that is *similar enough* to the *past cases* to be effective, but also *different enough* to address the *unique aspects* of the current problem. This step may involve selecting one or more past cases to use as a **starting point** for the solution, or it may involve **combining elements** from multiple past cases to create a new solution.

- **Revise- Evaluate the solution and revise it based on how well it works**

After a solution has been developed using past cases, the next step is to revise it to better fit the current problem. This may involve **modifying the solution based on feedback** from the user or on new information that has become available. The goal is to refine the solution to make it as effective as possible for the current problem. In some cases, the revision step may involve the use of machine learning algorithms to optimize the solution.

- **Retain- Decide whether to retain this new case in the case base**

The final step in the CBR process is to retain the newly developed solution for future use. This involves adding the new case to the case library so that it can be used in the retrieval step for future problems. The goal is to continually improve the quality of the case library and the effectiveness of the CBR process over time. The retention step may also involve the use of knowledge management tools to help organize and maintain the case library

Example:

- Help desk that users call with problems to be solved.
- When users give a description of a problem, the closest cases in the case base are retrieved.
- The diagnostic assistant could recommend some of these to the user, adapting each case to the user's particular situation.
- If one of the adapted cases works, that case is added to the case base, to be used when another user asks a similar question.
- If none of the cases found works, some other method is attempted to solve the problem, perhaps by adapting other cases or having a human help diagnose the problem.
- When the problem is finally solved, the solution is added to the case base.

Example

- Suppose that the database of a CBR system contains the following four cases:

Case	Monthly Income (£K)	Account Balance (£K)	Home Owner	Credit Score
1	3	2	0	2
2	2	1	1	2
3	3	2	2	4
4	0	-1	0	0

- The system is using the nearest neighbor retrieval algorithm with the following similarity function: $\boldsymbol{d(T,S) = \sum_{i=1}^{m} |T - S_i| * w_i}$
- where T is the target case, S is the source case, i is the number of a feature, and w_i are the weights. Cases with smaller values of d(T, S) are considered to be more similar.

Consider the following case:

Target Case (T):

Case	Monthly Income (£K)	Account Balance (£K)	Home Owner	Credit Score
5	3	1	2	?

Answer the following questions:

1. Which case the CBR system will retrieve as best match if all the weight w_i=1?
2. Suggest how solution of retrieved case can be adapted for target case.
3. What can be changed in the similarity function to make feature account balance three times more important than any other feature? Will this change influence the solution?

Case	Monthly Income (£K)	Account Balance (£K)	Home Owner	Credit Score
1	3	2	0	2
2	2	1	1	2
3	3	2	2	4
4	0	-1	0	0

Target Case (T):

Case	Monthly Income (£K)	Account Balance (£K)	Home Owner	Credit Score
5	3	1	2	?

$$d(T, S) = \sum_{i=1}^{m} |T_i - S_i| \times w_i$$

1. Which case the CBR system will retrieve as best match if all the weight w_i=1?

Feature	Case 1	Case 2	Case 3	Case 4
Monthly Income				
	|3-3|=0	|3-2|=1	|3-3|=0	|3-0|=3
Account Balance	|1-2|=1	|1-1|=0	|1-2|=1	|1-(-1)|=2
Home Owner	|2-0|=2	|2-1|=1	|2-2|=0	|2-0|=2
Total distance	**3**	**2**	**1**	**7**

✅ **Smallest distance = 1 (Case 3)**

Best match: Case 3

2. Suggest how solution of retrieved case can be adapted for target case.

- **Retrieved case:** Case 3
- **Case 3 Credit Score:** 4
- **Target case (Case 5):** Similar to Case 3, but has a lower account balance (1 instead of 2).

Adaptation reasoning:

Lower account balance might indicate slightly higher financial risk → slightly **lower credit score**.

So, **proposed Credit Score ≈ 3 or 3.5**

3. What can be changed in the similarity function to make feature account balance three times more important than any other feature? Will this change influence the solution?

To make *Account Balance* 3× more important:

$$w_{\text{Account Balance}} = 3, \text{ others } = 1$$

Recalculate distances:

Case	Income	Account Balance (×3)	Home Owner	Total Distance
1	0	1×3=3	2	**5**
2	1	0×3=0	1	**2**
3	0	1×3=3	0	**3**
4	3	2×3=6	2	**11**

✅ **Now smallest distance = 2 (Case 2)**

New best match: Case 2

Advantages of Case-Based Reasoning

- **Reusability**: CBR systems can reuse past solutions to similar problems, which can save time and effort compared to developing a solution from scratch.
- **Adaptability:** CBR systems can adapt to changing situations or contexts by selecting and modifying relevant cases.
- **Explanation**: CBR systems can provide explanations for their solutions based on the similar cases they retrieved.
- **Learning**: CBR systems can learn from new cases and refine their knowledge base over time.

Challenges of Case-Based Reasoning

- **Case Representation**: The quality of CBR depends on the accuracy and completeness of the cases used to solve a problem. If the cases are not well represented, it may lead to incorrect solutions.
- **Case Retrieval**: The success of CBR systems depends on the ability to retrieve relevant cases from the case base. If the retrieval process is not efficient or effective, it may lead to a poor solution.
- **Adaptation**: Adapting a retrieved case to a new problem domain can be difficult, as the retrieved case may not perfectly match the new problem.
- **Scalability:** As the size of the case base grows, the time required to retrieve and adapt cases can become significant, which can impact the efficiency of the system.

Applications of Case-Based Reasoning

It has been applied in various fields, including:

- Financial Decision Making: CBR systems can be used in financial institutions to help make decisions on loan approvals, risk assessments, and investment strategies by comparing past cases with current situations.
- Legal Reasoning: Case-Based Reasoning in machine learning systems can be used in the legal field to assist with case law research and the preparation of legal arguments by retrieving and adapting cases with similar legal issues.
- Transportation and Logistics: CBR systems can be used in transportation and logistics to optimize routing, scheduling, and resource allocation by learning from past cases.